

Master of Engineering

Hierarchical Action Models with 2D Paths and Consistency Flow Training for General Robot Manipulation

2D パス表現と一貫性フロー学習を用いた階層型生成モデルによる汎用的なロボット操作

OTA Naoto
太田 尚都Master's Program in Intelligent and Mechanical Interaction Systems
知能機能システム学位プログラム

Supervised by: TANAKA Fumihide, UTSURO Takehito and YAMAGUCHI Tomoyuki (Division of Intelligent Interaction Technologies)

指導教員: 田中 文英, 宇津呂 武仁, 山口 友之 (知能機能工学域)

Hierarchical vision-language-action (VLA) models that separate high-level semantic planning from low-level motion generation have emerged as a promising approach for robot manipulation. This paper integrates ManiFlow, a Consistency Flow Matching-based policy enabling high-quality action generation in 1-2 inference steps, as the low-level policy guided by 2D end-effector paths from a vision-language model (VLM) fine-tuned on robot path data. Through systematic experiments in RoboTwin 2.0 across 3 tasks and 6 conditions, we reveal three key findings. First, models trained on clean environments consistently outperform those trained on cluttered environments when evaluated on cluttered scenes (average success rate 20.3-28.0% vs 9.3-10.3%), suggesting that simplified training conditions facilitate learning essential motion patterns. Second, VLM path guidance improves performance in cross-domain settings (+7.7% average), indicating that semantic guidance can bridge domain gaps. Third, the Memory Function combining initial and current paths unexpectedly degrades performance, revealing that additional input information requires careful design. These results highlight both the potential and limitations of VLM-guided path conditioning for generalizable manipulation policies.

keywords: Robot Manipulation, Hierarchical Model, Consistency Flow Matching, Path Guidance, Generalization

1 Introduction

Developing general-purpose robot manipulation policies remains a long-standing challenge in robotics. Recent advances in large-scale vision-language models (VLMs) have spurred efforts to leverage their rich world knowledge for robot control ^{1, 2, 3, 4}. Vision-language-action (VLA) models, which directly fine-tune VLMs to predict robot actions, can generate control signals end-to-end from task descriptions and image observations, showing promise for generalization in open-world environments ^{2, 5, 6}.

However, such monolithic VLA models face several challenges. First, collecting high-quality robot manipulation data is costly, limiting the scale and diversity of available training data ^{7, 8, 9}. Second, VLA models have not yet achieved the level of classical robot control in terms of operational speed and precise motion control. On the other hand, smaller policy models can achieve more precise control compared to monolithic VLA models, but struggle to generalize to conditions different from their training environments ^{2, 10}. To address these challenges, hierarchical approaches that separate high-level semantic understanding by VLMs from precise motion generation by smaller policies have been proposed ^{7, 11, 12}.

HAMSTER, one such hierarchical approach, uses a VLM (VILA-1.5-13B) fine-tuned on robot path data to generate 2D end-effector paths, which serve as guidance for low-level policies such as RVT-2 or 3D Diffuser Actor ⁷. This design allows the VLM to focus on semantic trajectory planning while the low-level policy concentrates on precise control in 3D space. HAMSTER is also notable for its ability to leverage off-domain data without action labels, such as simulation data or data from different embodiments.

Meanwhile, generative models based on diffusion have demonstrated strong performance for robot manipulation ^{13, 14, 15}. However, diffusion models require numerous denoising steps during inference, posing challenges for real-time control. ManiFlow addresses this by adopting Consistency Flow Matching, enabling high-quality action generation in just 1-2 steps ¹⁶. ManiFlow employs the DiT-X (Diffusion Transformer with Cross-Attention) architecture to efficiently integrate diverse inputs including visual, language, and proprioceptive information.

In this work, we propose adopting a mechanism based on ManiFlow as the low-level policy within HAMSTER's hierarchical architecture and integrating 2D paths generated by the VLM into ManiFlow's input. While the original ManiFlow was evaluated under conditions

identical to training, we investigate whether introducing VILA path guidance can improve (1) single-task accuracy in the same environment, and (2) generalization performance to evaluation environments different from training.

Specifically, we propose two patterns for path input. The first pattern uses only the path prediction for the current frame. The second pattern additionally retains the path prediction from episode onset, using it alongside the current path. We refer to the latter as the Memory Function. This design aims to enable reference to stable past path information when current-frame path generation becomes unstable due to occlusion or other factors.

Evaluation experiments are conducted in the RoboTwin 2.0 simulation environment¹⁷⁾. We establish six experimental conditions by combining training environments (clean table or cluttered table) with evaluation environments (cluttered table), and quantitatively analyze the effects of path guidance on in-domain accuracy and cross-domain generalization performance.

2 Related Work

2.1 Hierarchical Robot Manipulation Policies

Hierarchical approaches in robot manipulation aim to solve complex tasks by separating high-level planning from low-level control. SayCan proposed combining the knowledge of large language models (LLMs) with robot affordance functions to generate action plans that are both executable and semantically appropriate¹¹⁾. Code as Policies demonstrated a hierarchical code generation approach that uses LLMs to generate program code for robot control, enabling complex spatial reasoning¹⁸⁾. Inner Monologue achieved closed-loop reasoning by providing environmental feedback to LLMs, integrating success detection and scene descriptions¹⁹⁾.

In hierarchical VLA models using VLMs, the choice of intermediate representation generated by the high-level model is an important design decision. Keypoint-based affordance prediction²⁰⁾ predicts manipulation target positions and placement locations as points, achieved through VLM fine-tuning. RT-Trajectory used 2D trajectory sketches as an intermediate representation to enable flexible task specification and policy generalization¹²⁾. For low-level policies, 3D-aware policies such as Perceiver-Actor²¹⁾ using voxelized 3D observations, and RVT²²⁾ and RVT-2²³⁾ using multi-view images from virtual viewpoints, have demonstrated high accuracy.

HAMSTER proposed a hierarchical architecture where a VLM predicts 2D end-effector paths, which serve as guidance for low-level 3D-

aware policies⁷⁾. A key advantage of this design is that the high-level VLM can be trained on off-domain data without action labels, such as simulation data or data from different embodiments. In HAMSTER, methods of overlaying path information on images and concatenating it as a separate input dimension were compared, with the latter showing superior performance. In this work, we adopt the method of overlaying paths onto images and using them as input to ManiFlow.

2.2 Diffusion Models and Flow Matching for Robot Control

Diffusion models are generative models that learn gradual transformations from noise to data, achieving significant success in image generation^{24, 25)}. Diffusion Policy applied diffusion models to robot visuomotor policies, demonstrating strong performance in representing multimodal action distributions and handling high-dimensional action spaces¹³⁾. 3D Diffusion Policy further improved visual generalization by using 3D representations extracted from sparse point clouds¹⁴⁾. 3D Diffuser Actor combined 3D scene representations with Diffusion Policy, achieving high performance on the RL Bench benchmark¹⁵⁾. However, diffusion models require numerous denoising steps during inference (typically 10-100 steps), posing challenges for real-time control applications.

Several approaches have been proposed to address this inference efficiency problem. DDIM, as a deterministic sampling variant of DDPM, enabled high-quality generation with fewer steps²⁶⁾. Consistency Models enabled single-step generation by learning direct mappings from arbitrary points in the diffusion process to target data²⁷⁾. For robot control applications, Consistency Policy proposed accelerating inference through knowledge distillation from pretrained Diffusion Policies²⁸⁾. ManiCM applied Consistency Models to 3D Diffusion Policy, achieving real-time 3D manipulation²⁹⁾.

Flow Matching, on the other hand, constructs generative models through a different approach than diffusion models. Flow Matching directly learns continuous transformations (flows) from noise distributions to data distributions in a simulation-free manner³⁰⁾. Rectified Flow achieved more efficient sampling by straightening these flows³¹⁾. These methods offer more stable training and higher-quality generation with fewer steps compared to diffusion models. π_0 combined VLA models with Flow Matching, achieving general control across diverse robot configurations including single-arm, bimanual, and mobile manipulators

32).

ManiFlow adopts Consistency Flow Matching, which integrates Flow Matching and Consistency Training¹⁶⁾. Specifically, in addition to the standard Flow Matching loss, it simultaneously optimizes a continuous-time Consistency loss that constrains different points on flow trajectories to converge to the same target data point. This design enables high-quality generation in 1-2 steps without requiring pretrained teacher models. The core of ManiFlow’s architecture, DiT-X, is built upon Diffusion Transformer³³⁾ and efficiently integrates diverse input modalities including visual, language, and proprioceptive information through cross-attention and AdaLN-Zero conditioning. In this work, we adopt ManiFlow as the low-level policy and integrate VLM-generated paths as additional visual input.

2.3 Domain Adaptation and Generalization

The domain gap between training and evaluation environments is a critical challenge when deploying robot manipulation policies to real environments. Domain Randomization, which randomizes rendering parameters in simulation environments, is widely used to facilitate transfer to real environments³⁴⁾. Colosseum is a benchmark that evaluates policy generalization through 20 tasks $\times$ 14 axes of environmental perturbations, measuring robustness to changes in color, texture, size, lighting, and background³⁵⁾.

Pretraining visual representations is also an effective approach for improving generalization. R3M pretrained visual representations using large-scale human video data, improving transfer performance to robot manipulation tasks through temporal contrastive learning and video-language alignment³⁶⁾. CLIP performed contrastive learning on 400 million image-text pairs, achieving zero-shot transfer capabilities³⁷⁾. DINOv2 performed self-supervised learning on 142 million images, providing general visual features without fine-tuning³⁸⁾.

RoboTwin 2.0 is a large-scale benchmark containing 731 objects, 50 tasks, and 5 robot embodiment types, implementing 5 axes of Domain Randomization including Clutter, Lighting, and Background¹⁷⁾. Compared to standard benchmarks such as RL Bench³⁹⁾ and MetaWorld⁴⁰⁾, it enables evaluation with more realistic visual variations. In this work, we use the RoboTwin 2.0 environment to evaluate how well policies trained on clean table conditions can generalize to cluttered table conditions. Whether VLM path guidance can bridge such domain gaps is one of the main questions investigated in this study.

3 Proposed Method

This chapter describes the proposed system that integrates 2D path guidance from VLMs into ManiFlow.

3.1 System Overview

The proposed system adopts a hierarchical architecture consisting of a high-level path generation module and a low-level action generation module. Figure 1 shows the overall system configuration.

The high-level module uses VILA-1.5-13B⁴¹⁾. VILA takes an RGB image and task description as input and outputs the 2D path that the end-effector should follow. The output path is represented as a sequence of normalized 2D coordinates (each coordinate in the range $[0, 1]$) and gripper states (open/close). We use the model fine-tuned on robot manipulation data from the HAMSTER project⁷⁾.

The low-level module uses ManiFlow¹⁶⁾. The original ManiFlow takes RGB images from the past 2 steps and the robot’s proprioceptive state (joint angles and gripper state) as input, generating 16 steps of actions at once. In this work, we overlay the 2D path generated by VILA onto these input images, thereby conveying high-level trajectory guidance to the low-level policy.

The proposed system provides two patterns for path input:

Pattern 1: Current path only

Only the path generated by VILA for the current frame is used. Specifically, the current path is overlaid onto each of the two past observation images that serve as input to ManiFlow, and provided to ManiFlow along with the robot state.

Pattern 2: Initial path + Current path (Memory Function)

Both the path generated at episode onset (initial path) and the path generated for the current frame (current path) are used. This design aims to enable reference to stable initial path information when current-frame path generation becomes unstable due to occlusion during task execution. We refer to this mechanism as the Memory Function.

The inference flow proceeds as follows. Since ManiFlow predicts 16 steps of actions at once, path generation occurs every 16 steps rather than every frame:

1. **Step 0:** Execute VILA path generation and create initial overlay and current overlay. Input these to ManiFlow and predict 16 steps of actions at once.
2. **Steps 1-15:** Sequentially execute the predicted actions.
3. **Step 16:** Execute new VILA path gener-

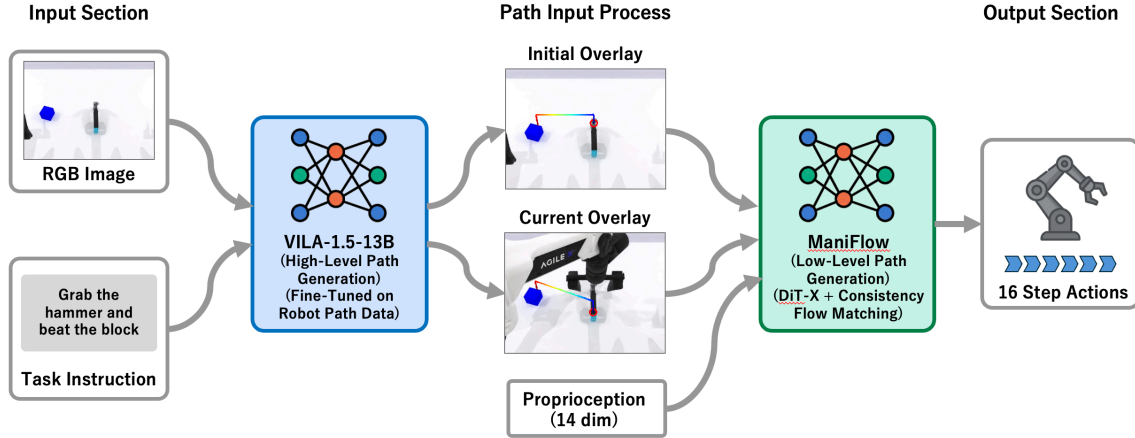


Figure 1 Overview of the proposed hierarchical system. The high-level module (VILA-1.5-13B) generates 2D end-effector paths from RGB images and task instructions. The generated paths are overlaid onto observation images and fed to the low-level module (ManiFlow), which outputs 16-step actions. Two path input patterns are supported: current path only, and initial path combined with current path (Memory Function).

ation and update the current overlay. The initial overlay remains fixed from episode onset. Predict the next 16 steps of actions at once.

4. Repeat every 16 steps until task completion.

This design limits the overhead of VILA path generation to once every 16 steps, minimizing the impact on real-time control.

3.2 High-Level Path Generation

The high-level module uses VILA-1.5-13B⁴¹⁾, which was fine-tuned on robot manipulation data in the HAMSTER project⁷⁾. VILA is a vision-language model combining the SigLIP visual encoder with the Vicuna language model, pretrained on 5.3 billion image-text pairs.

3.2.1 Input and Output

The input to VILA consists of an RGB image and a natural language task instruction. Task instructions are given in forms such as “there is a hammer and a block on the table, use the arm to grab the hammer and beat the block” or “click the bell’s top center on the table.”

VILA’s output is structured in the following format:

```
<ans>[(x1, y1), (x2, y2), <action>
  Close Gripper</action>, (x3, y3),
  ...]</ans>
```

Here, each coordinate (x, y) represents a 2D position normalized to the range $[0, 1]$, and `<action>` tags indicate gripper state change points. By parsing this output, we obtain a path representation including the gripper state

(open/close) corresponding to each waypoint.

3.2.2 Path Visualization

Generated paths are rendered as overlays on the original RGB image. The rendering specifications follow the original HAMSTER⁷⁾ implementation:

- **Path color:** Uses the jet colormap, encoding temporal progression through color changes from blue $\rightarrow$ cyan $\rightarrow$ green $\rightarrow$ yellow $\rightarrow$ red
- **Line width:** Scaled according to image size (based on 512×512 images)
- **Gripper markers:** Circular markers drawn only at gripper state change points
 - Open state: Blue circle (BGR: 255, 0, 0)
 - Close state: Red circle (BGR: 0, 0, 255)

Figure 2 shows examples of path overlays.

3.2.3 Path Generation Reliability

Since VILA is configured with `temperature=0`, the output is deterministic, but some frames may produce outputs that do not follow the specified format. To address this issue, we adopt the following strategies:

1. **Retry mechanism:** Up to 2 path generation attempts per frame
2. **Fallback parsing:** Automatic completion for outputs interpretable as coordinate data even when `<ans>` tags are omitted or coordinate parentheses are missing
3. **Fallback path:** When path generation fails, the most recently successful path is used as a substitute. For consecutive fail-

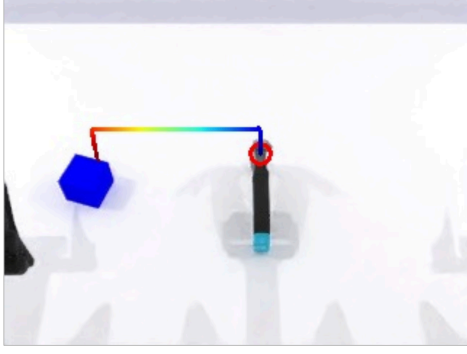


Figure 2 Examples of path overlays. Paths are rendered using the jet colormap (blue $\rightarrow$ cyan $\rightarrow$ green $\rightarrow$ yellow $\rightarrow$ red) to indicate temporal progression. Circular markers denote gripper state changes: blue for open, red for close.

ures at episode onset, the first successful path within the episode is retroactively applied

This design ensures the same fallback strategy is applied during both training and evaluation, maintaining consistency in the input distribution.

3.3 Low-Level Action Generation

The low-level module uses ManiFlow¹⁶⁾, a robot manipulation policy based on Consistency Flow Matching. This section describes ManiFlow’s architecture and how path input is integrated in this work.

3.3.1 ManiFlow Overview

ManiFlow achieves high-quality action generation in 1-2 steps through Consistency Flow Matching, which integrates Flow Matching³⁰⁾ and Consistency Training²⁷⁾. While conventional Diffusion Policy¹³⁾ requires 10-100 denoising steps during inference, ManiFlow achieves comparable or better quality with significantly fewer steps.

ManiFlow training simultaneously optimizes the standard Flow Matching loss and Consistency loss. The Flow Matching loss learns the velocity field from noise points to data points:

$$\mathcal{L}_{FM}(\theta) = \mathbb{E}_{x_0, x_1} [\|v_\theta(x_t, t) - (x_1 - x_0)\|^2] \quad (1)$$

Here, $x_t = (1 - t)x_0 + tx_1$ is the linear interpolation between noise x_0 and data x_1 . The Consistency loss constrains different points on flow trajectories to converge to the same target:

$$\mathcal{L}_{CT}(\theta) = \mathbb{E}_{t, \Delta t} [\|v_\theta(x_t, t, \Delta t) - \tilde{v}_{target}\|^2] \quad (2)$$

By combining these losses, high-quality generation is achieved in few steps without requiring pretrained teacher models.

3.3.2 DiT-X Architecture

The core of ManiFlow’s architecture is DiT-X (Diffusion Transformer with Cross-Attention). DiT-X is built upon Diffusion Transformer³³⁾ and features:

- **Adaptive cross-attention:** Enables fine-grained token-level interactions between action tokens and observation features (visual, language)
- **AdaLN-Zero conditioning:** Dynamically generates scale and shift parameters based on low-dimensional inputs such as timesteps and robot states, adaptively adjusting network behavior

These mechanisms enable efficient multi-modal integration of visual, language, and proprioceptive inputs.

3.3.3 Path Input Integration in This Work

The original ManiFlow takes RGB images from the past 2 steps and the robot’s proprioceptive state (14 dimensions: 7 DoF joint positions and gripper state for each arm) as input, generating 16 steps of actions at once.

In this work, we integrate high-level guidance from VILA by replacing the input RGB images with path overlay images. Table 1 shows the specific input configuration for each condition.

Path overlay images are the original RGB images with the 2D path generated by VILA visually rendered on them. ManiFlow is expected to learn the overlaid path information as visual features and utilize it for task execution.

3.4 Memory Function

In this work, we propose a second pattern for path input that combines the path from episode onset (initial path) with the current frame path (current path). We refer to this mechanism as the Memory Function.

During task execution, occlusion may occur when the robot’s end-effector or grasped objects obstruct the field of view. In such situations, the quality of paths generated by VILA may degrade, producing inaccurate or incomplete trajectories. At episode onset (frame 0), occlusion typically does not occur, and the task target objects are clearly visible. Therefore, paths generated at this point are considered reliable as overall trajectory plans for the task. The Memory Function maintains this initial path, enabling reference to overall task context information even when current path quality degrades during task execution.

When using the Memory Function, input to ManiFlow is configured as follows:

Table 1 Input Configuration for Each Condition

Condition	Input Image	Proprioception	Output
Original ManiFlow	RGB image (past 2 steps)	14 dim	16 steps of actions
Current path only	current overlay (past 2 steps)	14 dim	16 steps of actions
Initial + Current	initial overlay + current overlay	14 dim	16 steps of actions

- **Initial overlay:** RGB image at episode onset (frame 0) with the path generated at that time overlaid. Fixed until episode end.
- **Current overlay:** Current frame RGB image with the path generated for that frame overlaid. Updated every 16 steps.
- **Proprioceptive state:** Current robot state (14 dimensions)

Figure 3 shows a conceptual diagram of the Memory Function.

This mechanism is expected to improve occlusion robustness, preserve task context, and enable utilization of long-term planning information. Compared to using only the current path, this approach incurs slightly higher computational cost due to the additional input image, but offers the advantage of maintaining the overall plan from the initial state. In our experiments, we compare these two patterns and quantitatively verify the effectiveness of the Memory Function.

4 Experiments

This chapter describes the experimental setup and results for verifying the effectiveness of the proposed method.

4.1 Experimental Setup

Experiments were conducted in the RoboTwin 2.0 simulation environment¹⁷⁾. RoboTwin 2.0 is built on the SAPIEN 3.0.0b1 physics engine⁴²⁾ and provides benchmarks for bimanual manipulation tasks using the AgileX Cobot Magic robot. This work focuses on single-arm tasks, using the clean table and cluttered table conditions provided as table environment settings. In the cluttered table condition, objects unrelated to the task are randomly placed on the table, increasing visual complexity.

For evaluation tasks, we selected 3 tasks from RoboTwin 2.0’s single-arm tasks, considering difficulty and diversity of motion patterns. Click Bell involves simple contact motion, Move Can Pot is a standard pick-and-place task, and Beat Block Hammer requires tool use with two-stage motion. For each task, we collected 50 successful trajectories using RoboTwin 2.0’s scripted expert policy as training data.

In this work, we established 6 experimental conditions (C1-C6) by combining 3 model configurations (original ManiFlow, VILA + Mani-

Flow (current path), VILA + ManiFlow (initial + current path)) with 2 training environments (cluttered, clean). Table 2 shows the details of each condition. All conditions were evaluated on the cluttered table condition. Conditions C1-C3 are in-domain settings with training and evaluation on cluttered environments, while conditions C4-C6 are cross-domain settings with training on clean environments and evaluation on cluttered environments.

This design tests the following four hypotheses: **H1** (VILA path guidance improves in-domain accuracy), **H2** (VILA path guidance improves cross-domain generalization), **H3** (Memory Function outperforms current path only), and **H4** (path guidance effects are more pronounced in cross-domain conditions).

For evaluation metrics, we used task success rate based on 100 trials per task as the primary metric, with success determined using RoboTwin 2.0’s built-in judgment engine. As auxiliary metrics, we measured VILA path generation time, ManiFlow action generation time, and path generation success rate. ManiFlow training settings were unified across all conditions: 2 observation steps, 16 action prediction steps, and 501 training epochs.

4.2 Results and Analysis

4.2.1 Effect of Input Representation Mode

Figure 5 shows success rates by condition, and Table 3 provides detailed numerical results. This section verifies hypotheses H1-H4 regarding the effects of path input modes.

H1: Path Guidance Effect in In-domain Conditions

We compared C1 (baseline) with C2 and C3 (with path). C1→C2 showed -0.3% on average, and C1→C3 showed +0.7% on average, indicating no consistent improvement from VILA path guidance. This result suggests that path guidance may not provide additional information in in-domain conditions. Furthermore, the visual changes introduced by path overlays may have increased visual noise in cluttered training environments, making learning more difficult for ManiFlow.

H2: Path Guidance Effect in Cross-domain Conditions

We compared C4 (baseline) with C5 and C6 (with path). C4→C5 showed an average improvement of +7.7%, with improvements confirmed in 2 of 3 tasks (Click Bell: +17.0%, Beat

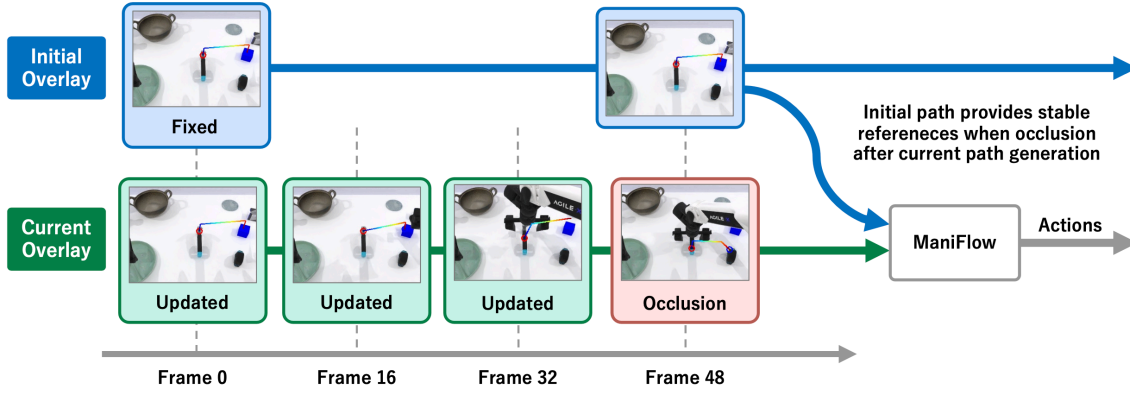


Figure 3 Memory Function overview. Initial overlay is fixed at frame 0, while current overlay updates every 16 steps. The initial path serves as stable reference when occlusion degrades current path quality.



Figure 4 Example scenes of the three evaluation tasks in RoboTwin 2.0 under cluttered table. (a) Click Bell, (b) Move Can Pot, (c) Beat Block Hammer.

Table 2 Experimental Conditions

Condition	Training Env	Evaluation Env	Path Input
C1	Cluttered	Cluttered	None (Original ManiFlow)
C2	Cluttered	Cluttered	Current path (Original ManiFlow with Path)
C3	Cluttered	Cluttered	Initial + Current path (Memory Function)
C4	Clean	Cluttered	None (Original ManiFlow)
C5	Clean	Cluttered	Current path (Original ManiFlow with Path)
C6	Clean	Cluttered	Initial + Current path (Memory Function)

Block Hammer: +6.0%). In contrast, C4→C6 showed a -7.3% decrease in performance. The partial effectiveness of path guidance in cross-domain conditions can be attributed to VILA’s paths compensating for the visual differences between training (clean) and evaluation (cluttered) environments. Since VILA is pretrained on large-scale data, it can generate appropriate paths even in cluttered environments, providing ManiFlow with trajectory information necessary for task execution.

H3: Effect of Memory Function (Initial + Current path)

We compared C2 vs C3 and C5 vs C6. In in-domain conditions (C2→C3), there was a slight improvement of +1.0% on average, but

in cross-domain conditions (C5→C6), a substantial performance decrease of -15.0% on average was observed. This contrasting result indicates that the effectiveness of initial path strongly depends on domain conditions. In in-domain conditions, the initial path may have functioned somewhat as contextual information for the overall task, contributing slightly to performance improvement. In cross-domain conditions, however, inputting two overlay images may have increased the amount of information to process, becoming a burden for models trained on clean environments. Additionally, as the episode progressed, information from the initial path (at frame 0) may have diverged from the current situation, providing mislead-

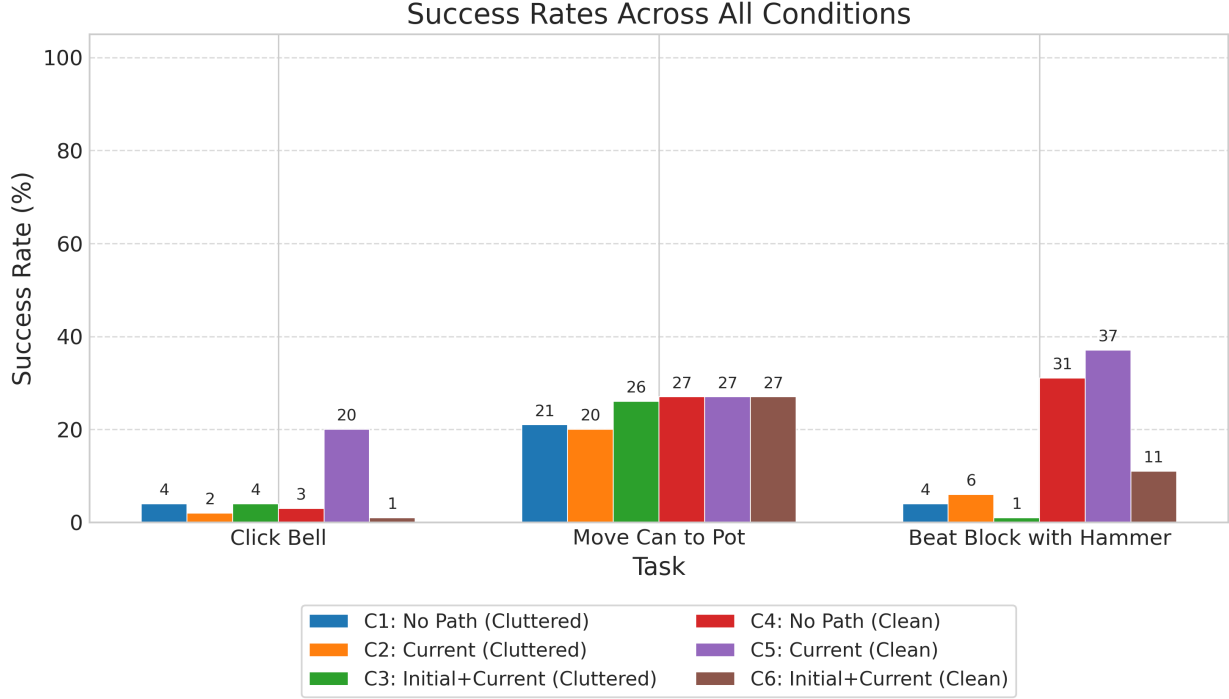


Figure 5 Success rate by condition

Table 3 Task Success Rate (%)

Task	C1	C2	C3	C4	C5	C6
Click Bell	4.0	2.0	4.0	3.0	20.0	1.0
Move Can Pot	21.0	20.0	26.0	27.0	27.0	27.0
Beat Block Hammer	4.0	6.0	1.0	31.0	37.0	11.0
Average	9.7	9.3	10.3	20.3	28.0	13.0

ing guidance. This result suggests that the effectiveness of Memory Function depends on the combination of training and evaluation environments.

H4: Domain Dependency of Path Guidance Effect

We compared (C2-C1) vs (C5-C4) and (C3-C1) vs (C6-C4). For the current path mode, the cross-domain improvement (+7.7%) was larger than the in-domain improvement (-0.3%), and this trend was confirmed in 2 of 3 tasks. In contrast, the initial+current mode showed the opposite pattern, with in-domain improvement (+0.7%) versus a substantial cross-domain decrease (-7.3%). This contrasting result indicates that the adaptation characteristics to domain gaps differ depending on the path input mode. The current path mode demonstrates effectiveness in cross-domain conditions where VILA's semantic guidance bridges the domain gap, while the initial+current mode is only effective in in-domain conditions and becomes counterproductive when domain gaps exist due to information inconsistency.

4.2.2 Effect of Training Environment

From the results in Table 3, an unexpected finding emerged that was not hypothesized. Conditions trained on clean environments (C4-C6, average 13.0-28.0%) consistently showed higher success rates than conditions trained on cluttered environments (C1-C3, average 9.3-10.3%). Particularly for the Beat Block Hammer task, C4 (31.0%) and C5 (37.0%) substantially outperformed C1-C3 (1.0-6.0%).

This result suggests that models trained on cluttered environments may have overfit to the scatter patterns specific to training data and failed to handle different scatter patterns during evaluation. In contrast, models trained on clean environments could efficiently learn essential motion patterns for tasks in environments with less visual noise, resulting in improved generalization to unseen cluttered environments.

4.2.3 Task-Specific Analysis

Different result patterns were observed across tasks. For Click Bell, C5 showed an outstanding success rate (20.0%), while other conditions remained low at 1.0-4.0%. For Move Can Pot, differences between conditions were relatively small (20.0-27.0%), indicating limited effectiveness of path guidance. For Beat Block Hammer, high success rates were achieved in C4 (31.0%) and C5 (37.0%), but other conditions remained low at 1.0-11.0%.

These results suggest that the effectiveness of VILA path guidance depends on task characteristics. For tasks requiring precise manipulation or clear trajectory planning (Click Bell, Beat Block Hammer), appropriately generated paths contribute to improved motion accuracy. In contrast, for standard pick-and-place tasks like Move Can Pot, motion patterns are relatively simple, limiting additional improvement from path guidance.

4.2.4 Training Dynamics

We conducted 501 epochs of training for all 18 conditions (3 tasks $\times$ 6 conditions) and analyzed the progression of Training Loss and Validation Loss. Figure 6 shows the Training Loss (top) and Validation Loss (bottom) for the six conditions on the Click Bell task.

For Training Loss, all six conditions converged along nearly identical curves, with no notable differences in training progression between conditions. This indicates that the presence or absence of path input and differences in training environments do not significantly affect the model's ability to fit the training data.

In contrast, Validation Loss showed different trends across conditions. While five conditions (C1, C2, C4, C5, C6) remained stable at low values throughout training, C3 (Cluttered training + Memory Function) exhibited an increasing trend as epochs progressed. This trend suggests signs of overfitting. This indicates that the combination of training on cluttered environments and the Memory Function may create conditions prone to overfitting.

Additionally, for the Beat Block Hammer task, Cluttered training conditions (C1-C3) showed a larger increase in Validation Loss after convergence compared to Clean training conditions (C4-C6). These results suggest that 50 episodes may not be sufficient training data for the increased input information from path overlays, Memory Function, and cluttered environments.

4.2.5 Computational Cost

Path generation by VILA incurs an overhead of approximately 4.3-4.7 seconds. Since ManiFlow alone has fast inference time of approx-

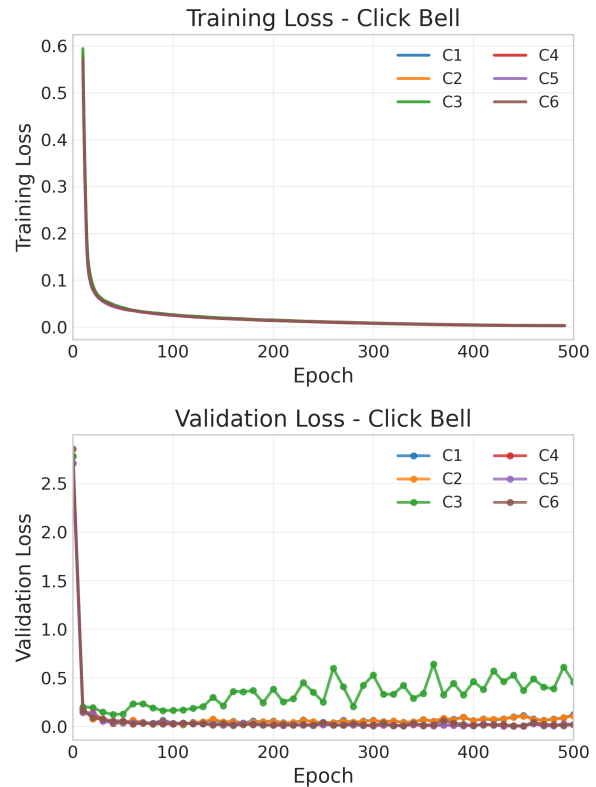


Figure 6 Training and Validation Loss curves for the Click Bell task across six conditions. Top: Training Loss. Bottom: Validation Loss.

imately 100-140ms, with the current design of VILA inference every 16 steps, overall episode execution time is approximately doubled. Considering C5's improvement (+7.7%) in cross-domain conditions, this overhead may be acceptable in application scenarios where generalization performance is important. For applications requiring real-time performance, lighter VLMs or optimization of path generation frequency is necessary.

4.2.6 Qualitative Analysis of Failure Cases

Qualitative analysis of failure episodes revealed three predominant failure patterns. First, the robot was observed to misidentify shadows on the table surface as target objects, causing it to move toward incorrect positions. Second, occlusion caused by the robot's end-effector or grasped objects during task execution degraded the accuracy of VILA's path generation, resulting in inaccurate trajectory overlays. Third, particularly in cluttered environments, the robot failed to correctly identify the target object among distractor objects and attempted to manipulate irrelevant items. These failure patterns were observed regardless of the presence or absence of path guidance, indicating fundamental challenges in robot manipulation

Table 4 Inference Time (ms)

Condition	VILA	ManiFlow	Total Overhead
C1, C4	-	108-136	-
C2, C5	4313-4703	118-136	+4.3-4.7s
C3, C6	4361-4363	118-126	+4.4s

under visually complex environments.

4.3 Limitations

This study has several limitations.

First, evaluations in this study were conducted only in the RoboTwin 2.0 simulation environment, without verification on real robots. A sim-to-real gap exists between simulation and real environments, and differences in visual appearance, physical properties, and sensor noise may affect performance. The extent to which VILA path guidance is robust to this sim-to-real gap needs to be verified through future real robot experiments.

Second, the tasks used for evaluation were limited to 3 types (Click Bell, Move Can Pot, Beat Block Hammer). While these tasks were selected considering difficulty and diversity of motion patterns, the effects of path guidance on more complex long-horizon tasks, bimanual coordination tasks, or contact-rich manipulation tasks remain unverified. Given that the effectiveness of path guidance was suggested to depend on task characteristics, evaluation on a broader set of tasks is necessary.

Third, regarding the unexpected performance degradation caused by the Memory Function (initial + current path), this study did not sufficiently elucidate its causes. The original design intent was to improve robustness by referencing the initial path when occlusion occurs. However, we did not quantitatively evaluate how much occlusion occurred in the tasks used in this experiment, nor how much it affected path generation quality. Future work should quantitatively analyze the frequency and impact of occlusion, and design more sophisticated Memory Function such as adaptive path reference based on occlusion detection.

5 Conclusion

In this work, we proposed a method that adopts ManiFlow as the low-level policy in HAMSTER’s hierarchical architecture and integrates 2D paths generated by VLMs into ManiFlow’s input. Through systematic experiments with 3 tasks $\times$ 6 conditions in the RoboTwin 2.0 simulation environment, we verified the effects of path guidance on single-task accuracy and generalization performance.

The experiments yielded the following findings. First, models trained on clean environments consistently showed higher general-

ization performance than models trained on cluttered environments (average success rate 20.3-28.0% vs 9.3-10.3%). This suggests that learning in visually simple environments promotes acquisition of essential motion patterns for tasks, consequently improving generalization to unseen environments. Second, VILA path guidance was effective in cross-domain conditions (clean training $\rightarrow$ cluttered evaluation), achieving an average +7.7% improvement in current path mode. This indicates the potential for VLM semantic guidance to bridge domain gaps. Third, the Memory Function (initial + current path) unexpectedly caused performance degradation, revealing that additional information is not necessarily beneficial in input representation design.

This study has clarified the potential and challenges of hierarchical approaches combining high-level trajectory planning by VLMs with fast action generation by Consistency Flow Matching. Future work should further verify the practicality of the proposed method through real robot verification, evaluation on a broader set of tasks, and design of Memory Function adaptive to occlusion.

Acknowledgments

I would like to show my sincere appreciation to Professor Fumihide Tanaka, Professor Takehito Utsuro, and Professor Tomoyuki Yamaguchi for their great help and invaluable academic guidance. I am also deeply grateful to Professor Dieter Fox and Mr. Ge Yan at the University of Washington for their exceptional support.

This work was supported by Cross-Pacific AI Initiative (X-PAI).

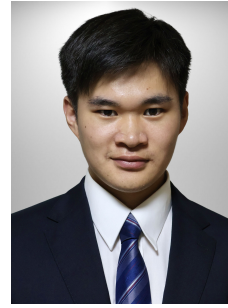
References

- 1) Brianna Zitkovich, Tianhe Yu, Sichun Xu, Peng Xu, Ted Xiao, Fei Xia, Jialin Wu, Paul Wohlhart, Stefan Welker, Ayzaan Wahid, et al. Rt-2: Vision-language-action models transfer web knowledge to robotic control. In *Conference on Robot Learning*, pages 2165–2183. PMLR, 2023.
- 2) Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan Foster, Grace Lam, Pannag Sanketi, et al. Openvla: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246*, 2024.

- 3) Danny Driess, Fei Xia, Mehdi S. M. Sajjadi, Corey Lynch, Aakanksha Chowdhery, Brian Ichter, Ayzaan Wahid, Jonathan Tompson, Quan Vuong, Tianhe Yu, Wenlong Huang, Yevgen Chebotar, Pierre Sermanet, Daniel Duckworth, Sergey Levine, Vincent Vanhoucke, Karol Hausman, Marc Toussaint, Klaus Greff, Andy Zeng, Igor Mordatch, and Pete Florence. Palm-e: an embodied multi-modal language model. In *Proceedings of the 40th International Conference on Machine Learning, ICML'23*. JMLR.org, 2023.
- 4) Oier Mees, Dibya Ghosh, Karl Pertsch, Kevin Black, Homer Rich Walke, Sudeep Dasari, Joey Hejna, Tobias Kreiman, Charles Xu, Jianlan Luo, et al. Octo: An open-source generalist robot policy. In *First Workshop on Vision-Language Models for Navigation and Manipulation at ICRA 2024*, 2024.
- 5) Abby O' Neill, Abdul Rehman, Abhiram Maddukuri, Abhishek Gupta, Abhishek Padalkar, Abraham Lee, Acorn Pooley, Agrim Gupta, Ajay Mandlekar, Ajinkya Jain, et al. Open x-embodiment: Robotic learning datasets and rt-x models: Open x-embodiment collaboration 0. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 6892–6903. IEEE, 2024.
- 6) Xinghang Li, Minghuan Liu, Hanbo Zhang, Cunjun Yu, Jie Xu, Hongtao Wu, Chilam Cheang, Ya Jing, Weinan Zhang, Huaping Liu, et al. Vision-language foundation models as effective robot imitators. *arXiv preprint arXiv:2311.01378*, 2023.
- 7) Yi Li, Yuquan Deng, Jesse Zhang, Joel Jang, Marius Memmel, Raymond Yu, Caelan Reed Garrett, Fabio Ramos, Dieter Fox, Anqi Li, et al. Hamster: Hierarchical action models for open-world robot manipulation. *arXiv preprint arXiv:2502.05485*, 2025.
- 8) Alexander Khazatsky, Karl Pertsch, Suraj Nair, Ashwin Balakrishna, Sudeep Dasari, Siddharth Karamcheti, Soroush Nasiriani, Mohan Kumar Srirama, Lawrence Yunliang Chen, Kirsty Ellis, Peter David Fagan, Joey Hejna, Masha Itkina, Marion Lepert, Yecheng Jason Ma, Patrick Tree Miller, Jimmy Wu, Suneel Belkhale, Shivin Dass, Huy Ha, Arhan Jain, Abraham Lee, Youngwoon Lee, Marius Memmel, Sungjae Park, Ilija Radosavovic, Kaiyuan Wang, Albert Zhan, Kevin Black, Cheng Chi, Kyle Beltran Hatch, Shan Lin, Jingpei Lu, Jean Mercat, Abdul Rehman, Pannag R Santheti, Archit Sharma, Cody Simpson, Quan Vuong, Homer Rich Walke, Blake Wulfe, Ted Xiao, Jonathan Heewon Yang, Arefeh Yavary, Tony Z. Zhao, Christopher Agia, Rohan Bajjal, Mateo Guaman Castro, Daphne Chen, Qiuyu Chen, Trinity Chung, Jaimyn Drake, Ethan Paul Foster, Jensen Gao, Vitor Guizilini, David Antonio Herrera, Minh Heo, Kyle Hsu, Jiaheng Hu, Muhammad Zubair Irshad, Donovan Jackson, Charlotte Le, Yunshuang Li, Kevin Lin, Roy Lin, Zehan Ma, Abhiram Maddukuri, Suvir Mirchandani, Daniel Morton, Tony Nguyen, Abigail O'Neill, Rosario Scalise, Derick Seale, Victor Son, Stephen Tian, Emi Tran, Andrew E. Wang, Yilin Wu, Annie Xie, Jingyun Yang, Patrick Yin, Yunchu Zhang, Osbert Bastani, Glen Berseth, Jeannette Bohg, Ken Goldberg, Abhinav Gupta, Abhishek Gupta, Dinesh Jayaraman, Joseph J Lim, Jitendra Malik, Roberto Martín-Martín, Subramanian Ramamoorthy, Dorsa Sadigh, Shuran Song, Jiajun Wu, Michael C. Yip, Yuke Zhu, Thomas Kollar, Sergey Levine, and Chelsea Finn. Droid: A large-scale in-the-wild robot manipulation dataset, 2025.
- 9) Homer Walke, Kevin Black, Abraham Lee, Moo Jin Kim, Max Du, Chongyi Zheng, Tony Zhao, Philippe Hansen-Estruch, Quan Vuong, Andre He, Vivek Myers, Kuan Fang, Chelsea Finn, and Sergey Levine. Bridgedata v2: A dataset for robot learning at scale, 2024.
- 10) Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Joseph Dabis, Chelsea Finn, Keerthana Gopalakrishnan, Karol Hausman, Alex Herzog, Jasmine Hsu, et al. Rt-1: Robotics transformer for real-world control at scale. *arXiv preprint arXiv:2212.06817*, 2022.
- 11) Alex Irpan, Alexander Herzog, Alexander Toshkov Toshev, Andy Zeng, Anthony Brohan, Brian Andrew Ichter, Byron David, Carolina Parada, Chelsea Finn, Clayton Tan, et al. Do as i can, not as i say: Grounding language in robotic affordances. *arXiv preprint arXiv:2204.01691*, 2022.
- 12) Jiayuan Gu, Sean Kirmani, Paul Wohlhart, Yao Lu, Montserrat Gonzalez Arenas, Kanishka Rao, Wenhao Yu, Chuyuan Fu, Keerthana Gopalakrishnan, Zhuo Xu, et al. Rt-trajectory: Robotic task generalization via hindsight trajectory sketches. *arXiv preprint arXiv:2311.01977*, 2023.
- 13) Cheng Chi, Zhenjia Xu, Siyuan Feng, Eric Cousineau, Yilun Du, Benjamin Burchfiel, Russ Tedrake, and Shuran Song. Diffusion policy: Visuomotor policy learning via ac-

- tion diffusion. *The International Journal of Robotics Research*, 44(10-11):1684–1704, 2025.
- 14) Yanjie Ze, Gu Zhang, Kangning Zhang, Chenyuan Hu, Muhan Wang, and Huazhe Xu. 3d diffusion policy: Generalizable visuomotor policy learning via simple 3d representations. *arXiv preprint arXiv:2403.03954*, 2024.
 - 15) Tsung-Wei Ke, Nikolaos Gkanatsios, and Katerina Fragkiadaki. 3d diffuser actor: Policy diffusion with 3d scene representations, 2024.
 - 16) Ge Yan, Jiyue Zhu, Yuquan Deng, Shiqi Yang, Ri-Zhao Qiu, Xuxin Cheng, Marius Memmel, Ranjay Krishna, Ankit Goyal, Xiaolong Wang, et al. Manifold: A general robot manipulation policy via consistency flow training. *arXiv preprint arXiv:2509.01819*, 2025.
 - 17) Tianxing Chen, Zanxin Chen, Baijun Chen, Zijian Cai, Yibin Liu, Zixuan Li, Qiwei Liang, Xianliang Lin, Yiheng Ge, Zhenyu Gu, et al. Robotwin 2.0: A scalable data generator and benchmark with strong domain randomization for robust bimanual robotic manipulation. *arXiv preprint arXiv:2506.18088*, 2025.
 - 18) Jacky Liang, Wenlong Huang, Fei Xia, Peng Xu, Karol Hausman, Brian Ichter, Pete Florence, and Andy Zeng. Code as policies: Language model programs for embodied control. *arXiv preprint arXiv:2209.07753*, 2022.
 - 19) Wenlong Huang, Fei Xia, Ted Xiao, Harris Chan, Jacky Liang, Pete Florence, Andy Zeng, Jonathan Tompson, Igor Mordatch, Yevgen Chebotar, Pierre Sermanet, Noah Brown, Tomas Jackson, Linda Luu, Sergey Levine, Karol Hausman, and Brian Ichter. Inner monologue: Embodied reasoning through planning with language models, 2022.
 - 20) Wentao Yuan, Jiafei Duan, Valts Blukis, Wilbert Pumacay, Ranjay Krishna, Adithyavairavan Murali, Arsalan Mousavian, and Dieter Fox. Robopoint: A vision-language model for spatial affordance prediction for robotics. *arXiv preprint arXiv:2406.10721*, 2024.
 - 21) Mohit Shridhar, Lucas Manuelli, and Dieter Fox. Perceiver-actor: A multi-task transformer for robotic manipulation. In *6th Annual Conference on Robot Learning*, 2022.
 - 22) Ankit Goyal, Jie Xu, Yijie Guo, Valts Blukis, Yu-Wei Chao, and Dieter Fox. Rvt: Robotic view transformer for 3d object manipulation, 2023.
 - 23) Ankit Goyal, Valts Blukis, Jie Xu, Yijie Guo, Yu-Wei Chao, and Dieter Fox. Rvt-2: Learning precise manipulation from few demonstrations, 2024.
 - 24) Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. *Advances in neural information processing systems*, 33:6840–6851, 2020.
 - 25) Yang Song, Jascha Sohl-Dickstein, Diederik P. Kingma, Abhishek Kumar, Stefano Ermon, and Ben Poole. Score-based generative modeling through stochastic differential equations, 2021.
 - 26) Jiaming Song, Chenlin Meng, and Stefano Ermon. Denoising diffusion implicit models. In *International Conference on Learning Representations*, 2021.
 - 27) Yang Song, Prafulla Dhariwal, Mark Chen, and Ilya Sutskever. Consistency models, 2023.
 - 28) Aaditya Prasad, Kevin Lin, Jimmy Wu, Linqi Zhou, and Jeannette Bohg. Consistency policy: Accelerated visuomotor policies via consistency distillation. *arXiv preprint arXiv:2405.07503*, 2024.
 - 29) Guanxing Lu, Zifeng Gao, Tianxing Chen, Wenxun Dai, Ziwei Wang, Wenbo Ding, and Yansong Tang. Manim: Real-time 3d diffusion policy via consistency model for robotic manipulation. *arXiv preprint arXiv:2406.01586*, 2024.
 - 30) Yaron Lipman, Ricky TQ Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. *arXiv preprint arXiv:2210.02747*, 2022.
 - 31) Xingchao Liu, Chengyue Gong, and Qiang Liu. Flow straight and fast: Learning to generate and transfer data with rectified flow. *arXiv preprint arXiv:2209.03003*, 2022.
 - 32) Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, et al. π_0 : A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024.
 - 33) William Peebles and Saining Xie. Scalable diffusion models with transformers. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 4195–4205, 2023.
 - 34) Josh Tobin, Rachel Fong, Alex Ray, Jonas Schneider, Wojciech Zaremba, and Pieter Abbeel. Domain randomization for transferring deep neural networks from simulation to the real world. In *2017 IEEE/RSJ international conference on intelligent robots and systems (IROS)*, pages 23–30. IEEE, 2017.
 - 35) Wilbert Pumacay, Ishika Singh, Jiafei Duan, Ranjay Krishna, Jesse Thomason, and Dieter Fox. The colosseum:

- A benchmark for evaluating generalization for robotic manipulation. *ArXiv*, abs/2402.08191, 2024.
- 36) Suraj Nair, Aravind Rajeswaran, Vikash Kumar, Chelsea Finn, and Abhinav Gupta. R3m: A universal visual representation for robot manipulation. *arXiv preprint arXiv:2203.12601*, 2022.
 - 37) Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. Learning transferable visual models from natural language supervision. In *International Conference on Machine Learning*, 2021.
 - 38) Maxime Oquab, Timothée Darcet, Théo Moutakanni, Huy V. Vo, Marc Szafraniec, Vasil Khalidov, Pierre Fernandez, Daniel HAZIZA, Francisco Massa, Alaaeldin El-Nouby, Mido Assran, Nicolas Ballas, Wojciech Galuba, Russell Howes, Po-Yao Huang, Shang-Wen Li, Ishan Misra, Michael Rabbat, Vasu Sharma, Gabriel Synnaeve, Hu Xu, Herve Jegou, Julien Mairal, Patrick Labatut, Armand Joulin, and Piotr Bojanowski. DINOv2: Learning robust visual features without supervision. *Transactions on Machine Learning Research*, 2024. Featured Certification.
 - 39) Stephen James, Zicong Ma, David Rovick Arrojo, and Andrew J. Davison. Rlbench: The robot learning benchmark & learning environment. *IEEE Robotics and Automation Letters*, 5:3019–3026, 2019.
 - 40) Tianhe Yu, Deirdre Quillen, Zhanpeng He, Ryan C. Julian, Karol Hausman, Chelsea Finn, and Sergey Levine. Meta-world: A benchmark and evaluation for multi-task and meta reinforcement learning. *ArXiv*, abs/1910.10897, 2019.
 - 41) Ji Lin, Hongxu Yin, Wei Ping, Pavlo Molchanov, Mohammad Shoeybi, and Song Han. Vila: On pre-training for visual language models. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 26689–26699, 2024.
 - 42) Fanbo Xiang, Yuzhe Qin, Kaichun Mo, Yikuan Xia, Hao Zhu, Fangchen Liu, Minghua Liu, Hanxiao Jiang, Yifu Yuan, He Wang, et al. Sapien: A simulated part-based interactive environment. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 11097–11107, 2020.



Naoto Ota

Master' s Program
in Intelligent and Me-
chanical Interaction
Systems, University of
Tsukuba.